20 确定性、精度与性能验证

> 目标：学会验证优化有没有改变不该改变的结果，并区分“快了”“更准了”“能重放了”这三种不同结论。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 本章的源码预期值来自静态核查；数值例为理论推导，不表示本次已经运行 C 测试或性能测量。

20.1 先定义成功，才能讨论优化

最朴素、也最容易比较的基线是：

- 固定创建与销毁顺序。
- 固定每帧输入。
- 固定时间步与子步数。
- 单线程执行。
- 保持同一编译配置。

以后每增加一个优化，先问它应保持什么不变。

验收目标	问题
物理正确性	接触方向、冲量符号、约束限制是否正确?
数值精度	与参考值或可接受误差相比偏差多大?
确定性	同条件相同输入是否逐位得到同结果?
性能	相同任务和质量要求下消耗多少时间、内存?

一个总是稳定算错的求解器可以非常确定。 一个更接近数学真值的 FMA 路径，也可能与另一平台产生不同末位。 只观察两个画面相似，不能证明其中任何一项。

还有一些优化本来就是受控近似。 例如 Contact recycling 改变了 manifold 刷新的时机。 比较开启与关闭回收的结果，应检查误差与业务行为，而不是预设两者必须拥有完全相同 hash。

20.2 并行计算，串行按稳定键提交

朴素并行方案常让 worker 算完一个接触就立即追加结果。 追加时加锁，可以保护容器不被写坏，却不能固定哪个 worker 先拿到锁。

这会沿着如下链条影响物理结果：

1. 完成顺序改变接触创建顺序。
2. 创建顺序可能改变接触 ID 和数组布局。
3. 布局与插入顺序影响图着色。

4. 颜色和约束顺序改变浮点迭代顺序。
5. 后续位置、接触和事件逐渐分叉。

所以当前实现并不是“所有数组最后统一排序”。 它在关键状态提交位置使用稳定索引或位图，将计算完成次序与提交次序分开。

接触状态位图

窄阶段每个 worker 拥有自己的 `contactStateBitSet`。 发生开始触碰、停止触碰或完全分离时，设置对应 `contactId` 的位。 这里使用持久 contact ID，而不是会迁移的 contact Sim 指针。

worker 完成以后：

1. 将所有位图按位或到一个集合。
2. 按 64 位字块索引递增遍历。
3. 每次通过 `ctz` 找最低置位。
4. 计算 `contactId = 64 * block + bit`。
5. 串行执行接触迁移、岛连接、着色和事件提交。

例如 worker 0 得到 $\{70, 2\}$，worker 1 得到 $\{9, 2\}$。 最终提交顺序是 2、9、70，重复的 2 只处理一次。 这个顺序不依赖谁先完成。

```
// Process contact state changes. Iterate over set bits
for ( uint32_t k = 0; k < bitSet->blockCount; ++k )
{
	uint64_t bits = bitSet->bits[k];
	while ( bits != 0 )
	{
		uint32_t ctz = b2CTZ64( bits );
		int contactId = (int)( 64 * k + ctz );
```

处理后使用 `bits & (bits - 1)` 清除最低置位，继续下一个 ID。 这是一项明确的稳定枚举协议，不是抽象地“排一下序就确定了”。

位图不是零成本

令 ID 容量 $N = 100000$，每块 64 位，每块 8 B。 每个 worker 的位图大小为：

$$\left\lceil \frac{100000}{64} \right\rceil \times 8 = 1563 \times 8 = 12504 \text{ B}.$$

8 个 worker 合计为 100,032 B，不包含结构体和分配器开销。 归约还需要约 7×1563 次 64 位 OR，以及清零和枚举成本。 当 ID 容量大而变化很少时，这些扫描也值得测量。

实验：固定输入，把 worker 数改为 1、2、4、8，逐帧比较 hash。 若首先发现接触创建或着色顺序不同，应先检查提交协议，而不是只调浮点容差。

20.3 宽阶段的确定性不是按原子分配顺序

并行配对会为每个 moved proxy 保留自己的 `moveResults` 槽。 候选 pair 的内存空间可以通过原子索引分配，但创建接触时不按这些分配地址遍历。

`b2UpdateBroadPhasePairs` 按 `moveArray` 顺序，依次消费每个查询自己的结果链表。 因此原子分配先后不直接变成接触创建先后。 前提是移动数组和每个查询的树遍历本身保持相应稳定性。

`FinalizeBodies` 对 enlarged body 的处理又用了另一种位图。 它按 `bodySimIndex` 标记，而非 contact ID。 这些稠密下标在该阶段内有效，所有 worker 位图 OR 后按索引顺序建立移动代理数组。

为什么睡眠要在后面？ 因为睡眠会 RemoveSwap 醒体数组，使旧 `bodySimIndex` 失效。 确定性不仅是排序问题，也包括索引在哪个生命周期里仍然有效。

fast bullet 的移动代理在非确定顺序 CCD 工作之前先进入确定的移动数组。 后续某些 bullet 列表遍历允许非确定顺序，是因为该操作被设计为不让顺序影响结果。 不能反过来推导“所有并行临时数组都必须排序”。

岛拆分候选还有显式平局规则：睡眠时间相等时选择较大的岛 ID。 同色约束只写互不共享的动态端，色间屏障则固定跨色依赖顺序。 这些协议共同维护确定性，没有一个单独的“确定性开关”能替代它们。

20.4 浮点数的间距可以手算

对于二进制 32 位浮点正常数，若数值位于 $[2^e, 2^{e+1})$，相邻表示值间距为：

$$\mathrm{ULP}(x) = 2^{e-23}.$$

它表示这个数量级附近的分辨率，不是一个复杂算法的完整误差上界。

位置量级	所在二进制区间	单精度间距
$10^5\ \mathrm{m}$	$[2^{16}, 2^{17})$	0.0078125 m
$10^6\ \mathrm{m}$	$[2^{19}, 2^{20})$	0.0625 m
$10^7\ \mathrm{m}$	$[2^{23}, 2^{24})$	1 m

在 $10^6\ \mathrm{m}$ 附近给一个可表示坐标增加 1 cm，增量可能在舍入时完全消失。 问题不是积分公式没有执行，而是结果落回了同一个浮点表示值。

这种误差会进入接触间距、关节锚点差和世界位置更新。 不能只把最终渲染坐标改成 double，就认为物理阶段已经恢复丢失的信息。

20.5 算式相同，不代表计算路径相同

浮点加法一般不满足结合律。 用无量纲标量 $a = 10^8$、$b = -10^8$、$c = 1$ 举例：

$$\mathrm{fl}(\mathrm{fl}(a + b) + c) = 1,$$

$$\mathrm{fl}(a + \mathrm{fl}(b + c)) = 0.$$

第二条路径中，$b + c$ 的 1 先被舍入丢失。 这说明即使没有数据竞争，改变约束计算次序也可能改变结果。

FMA 为什么又准又可能不同

设三个无量纲、可由 float 表示的数：

$$a = 1 + 2^{-23}, \quad b = 1 - 2^{-23}, \quad c = -1.$$

精确乘积为：

$$ab = 1 - 2^{-46}.$$

分开的乘法先把乘积舍入成 1，再加上 -1，结果为 0。 FMA 将乘加作为一次舍入，得到 -2^{-46}。

FMA 在这个例子中更接近精确值，却不等于另一条路径的结果。 所以跨平台确定性有时要求约束编译器的运算收缩行为。

当前顶层 CMake 对 GNU、Clang 使用 `-ffp-contract=off`。 clang-cl 则通过 `/clang:-ffp-contract=off` 传递到前端。 这是按编译器判断，不是只在某个操作系统才启用。 不要擅自加入 fast-math 后继续沿用原确定性承诺。

数学函数也属于配置合同

当前 `b2Atan2` 使用自定义近似表达式。 `b2ComputeCosSin` 使用角度归约、有理式近似和归一化。 它们不是简单调用系统 `atan2f/sinf/cosf` 后假定所有库相同。

这不意味着“所有浮点问题都被消灭”。 例如实现仍使用 `sqrtf`，实际支持范围还需要编译配置和测试来验证。 数值精度测试与确定性回归测试应分别保留。

20.6 双精度世界坐标，不等于整个求解器 double

朴素全 float 容易在远离原点时丢失小距离。 朴素全 double 则可能把大量局部向量、约束与热数据一并扩大。 当前采用混合精度边界。

数据	大世界模式下的精度
`b2Pos` 世界平移	double
`b2WorldTransform.p`	double
`b2WorldTransform.q`	float
`b2Vec2`、局部几何	float
线速度、角速度	float
求解增量	float
`b2AABB`	float

启用宏为 `BOX2D_DOUBLE_PRECISION`。 该宏通过 CMake 作为 PUBLIC 定义传递，因为库和调用者必须看到一致的头文件布局。 只改应用一侧宏会造成 ABI 不匹配。

关键是先相减，再降精度

令：

$$x_A = 10000000.0\ \mathrm{m}, \quad x_B = 10000000.25\ \mathrm{m}.$$

若先分别转成 float，再相减，两个数可能都变成 10000000，差为零。 若先用 double 相减，再转为 float，则得到可精确表示的 0.25 m。

```
/// a - b, demoted to float. The primary precision boundary operation.
B2_INLINE b2Vec2 b2SubPos( b2Pos a, b2Pos b )
{
	return B2_LITERAL( b2Vec2 ){ (float)( a.x - b.x ), (float)( a.y - b.y ) };
}
```

`b2InvMulWorldTransforms` 同样先计算世界平移差，再进入 A 的局部坐标系。 `b2UpdateContact` 在 A frame 中调用 manifold 算法，随后恢复世界朝向与相对锚点。

求解器还使用大步内的 `deltaPosition` 和 `deltaRotation`，避免每个子步直接在巨大绝对位置上累计微小变化。 大步结束再通过 `b2OffsetPos` 更新世界质心。

例如 0.001 m 附近 float 间距约为 $2^{-33} \approx 1.16 \times 10^{-10}\ \mathrm{m}$。 这比在 100 km 附近直接修改世界 float 坐标更适合表示小增量。 但局部浮点计算仍有误差，相对坐标不是精确算术。

20.7 为什么大世界的宽阶段仍可能变慢

世界位置可以 double，树的 AABB 仍是 float。 把 double 边界转换成 float 时必须保守向外舍入。 否则包围盒可能向内缩，导致真实候选被漏掉。

当前辅助函数分别向负无穷和正无穷取边界。 必要时使用 `nextafterf` 移到相邻 float。 几何 AABB 先在 double 位置上加入额外范围，再执行向外转换。

例如真实 x 区间为：

$$[10000000.1, 10000000.2]\ \mathrm{m}.$$

保守 float 区间可以变成：

$$[10000000, 10000001]\ \mathrm{m}.$$

实际宽度 0.1 m，被扩大成了 1 m。 这种转换避免漏碰撞，却增加了假阳性候选。 所以大世界模式不是“无限远、无限小物体、零性能代价”的承诺。

`b2ToVec2` 明确是有损转换。 如果业务先把远世界坐标降成 float 再交给引擎，后面无法恢复已丢失的信息。

实验：在原点和 x=10^7 m 处建立相同相对布局。 比较相对位置、命中数量和查询 fraction，同时记录候选访问数。 精度通过不代表树查询成本不变。

20.8 哪些测试使用容差，哪些必须精确相等

当前大世界金字塔测试在 double 模式下比较原点与 10^7 m 场景。 它要求睡眠步号相等，相对位置差使用 0.001 m 容差。 大世界查询测试对命中数量使用精确比较，对 fraction 和位置分别使用容差。

这些是在验证平移后的几何精度，使用 epsilon 是有意义的。 但不能据此说“Box2D 确定性测试只检查 epsilon”。

`test/test_determinism.c` 对睡眠步号和最终 hash 使用精确相等。 当前写在源码中的预期值为：

编译精度	预期 sleepStep	预期 hash
float 世界位置	245	`0x9F027FDF`
double 世界位置	356	`0x448C8E48`

这些值是当前提交的回归基准，不是本章作者本次运行得出的结果。 两种精度的预期不同，说明不能要求 float 与 double 使用同一个 golden hash。

`MultithreadingTest` 重复不同 worker 数，`BuiltInSchedulerTest` 检查内置调度，`CrossPlatformTest` 使用固定预期回归。 改变算法后若 hash 变化，应先分析原因，再决定是否有依据更新基准。 不能把更新预期值当作“修复确定性测试”。

20.9 输入录制、快照和状态 hash 各有职责

输入录制回答“用户按什么次序调用了哪些影响仿真的操作”。 快照回答“怎样从一个中途状态继续仿真”。 状态 hash 回答“两个执行过程在某个检查点是否出现可检测差异”。

只保存刚体位置和速度通常不足以无缝回放。 持久接触冲量、睡眠集合、ID 池、约束布局和树状态都可能影响下一步。 两份画面相同的世界，内部状态可以不同。

当前存在三类容易混淆的 hash：

入口	主要覆盖内容
Falling Hinges 测试中的 hash	按创建 body ID 顺序混入最终 `b2WorldTransform` 字节
`b2HashWorldState`	按活跃身份槽顺序混入位姿，以及存在 State 时的速度
`b2HashWorldStateDeep`	再加入部分索引、接触和关节冲量、ID 池统计等

浅 hash 不包含整个世界。 Deep hash 也不等于逐字段覆盖全部世界，例如它没有混入所有树节点和所有 manifold 几何字段。 公共 `b2World_GetStateHash` 使用 deep 路径。

hash 不同可以确定观察到的状态不一致。 hash 相同则仍存在覆盖范围和哈希碰撞的局限，不能当数学证明。

录制每步的 StateHash 时，当前世界仍保持 locked。 这样可以维持录制缓冲的写入协议，不在解锁后与其他操作竞争追加。

实验：逐帧比较而不是只比较最后一帧。 找到第一次分叉，再比较该帧前后的接触 ID、集合索引、冲量、位姿和速度。 这比看两段最终动画哪里不同更容易定位原因。

20.10 快照是续算工具，不是任意版本通用档案

当前快照头包含 magic、version、layout hash 和精度标志。 加载时拒绝不兼容精度与布局。 layout hash 混入多个关键结构大小和常量，也包括指针大小。

但同尺寸字段交换或含义重解释，未必改变结构大小。 因此格式版本仍需要人为维护，不能认为 layout hash 自动解决所有兼容性问题。

快照 API 使用两步缓冲模式：

1. `b2World_Snapshot(world, NULL, 0)` 查询需要大小。
2. 分配足够缓冲，再调用填写。

实际提供缓冲但容量不足时，函数返回所需大小，不复制超容量数据。 不能把返回值解释成“永远等于成功写入字节数”。

快照和恢复必须发生在世界未锁定时。 新建世界加载快照后，不携带原来的主机回调和用户指针，应由业务重新接线。 原地 Restore 保留世界身份与主机连接，但需要注意失败边界。

头部检查失败可以在不修改世界的情况下返回。 然而通过头部检查后，截断或损坏的主体仍可能在开始重建后失败；当前注释要求调用者销毁已经不可用的世界。 不要把所有失败都描述成事务式“原世界完全不变”。

已有 `SnapshotTest` 不只比较两份克隆是否互相一致。 它还从运动中的真实世界做快照，保存真实世界后续 90 步 hash，再让加载世界逐步复现。 这比“两份克隆以相同方式算错”更有检验价值。

20.11 现有测试入口怎样使用

测试可执行文件的第一个参数是精确测试函数名。 它不是子串过滤，也不是 `--filter=...`。 不传参数则运行全部注册测试。

下列为供读者执行的命令示例，非源码；本任务没有执行这些命令。 假设使用 Windows 多配置构建，输出位于 `build-perf/bin/Release`：

```
& .\build-perf\bin\Release\test.exe DeterminismTest
& .\build-perf\bin\Release\test.exe SnapshotTest
& .\build-perf\bin\Release\test.exe LargeWorldTest
& .\build-perf\bin\Release\test.exe DynamicTreeTest
& .\build-perf\bin\Release\test.exe ThreadTest
```

对应功能为：

测试	重点
`DeterminismTest`	worker、内置调度和固定精确预期
`SnapshotTest`	睡眠集合、恢复、持有 ID、续算和录制
`LargeWorldTest`	远坐标几何、查询与回放
`DynamicTreeTest`	查询、移动、重建及树结构
`ThreadTest`	线程相关行为

还有 `RecordingTest`、`RecordingKeyframeTest`、`RecordingScrubTest`、`ReStepRaceTest` 等独立入口。传参拼错时不能只看到程序结束就认为目标测试通过，应检查实际输出的测试名。

当前单元测试需要静态链接。 顶层 CMake 在 `BUILD_SHARED_LIBS=ON` 时会跳过该测试目标。 这里的测试目标名为 `test`，不是为每个函数单独创建一个可执行文件。 当前所查 CMake 文件也没有把这些函数注册成对应的 CTest 测试名。

20.12 建立可复现的性能基线

先记录以下条件，再看耗时数字：

- 提交 ID、编译器和版本。
- CPU、操作系统、架构与功耗环境。
- Release、Debug 或 RelWithDebInfo。
- SIMD、AVX2、double 和验证配置。
- worker 数、时间步、子步数。
- 场景创建顺序和随机输入。
- 睡眠、CCD、热启动、回收等功能开关。

60 Hz 的一帧预算为：

$$1000/60 \approx 16.667\ \text{ms}.$$

如果假设测得物理 10 ms、其他工作 6 ms，总体已接近预算。 接着应看物理内部是配对、窄阶段、求解还是维护占主导，而不是凭术语选择 SIMD。 这个数字例只是预算演算，不是仓库实测报告。

`b2Profile` 含嵌套和不同范围的计时字段。 不能把 `solve`、`constraints` 等重叠区间全部加起来当总时间。 先读计时位置，再解释字段。

构建命令示例，非源码，未在本任务执行：

```
cmake -S . -B build-perf -DBOX2D_SAMPLES=OFF -DBOX2D_UNIT_TESTS=ON -DBOX2D_BENCHMARKS=ON -
DBUILD_SHARED_LIBS=OFF
cmake --build build-perf --config Release --target test benchmark
```

单配置生成器的可执行文件路径通常不带 `Release` 子目录，应以自己的构建输出为准。 不同配置建议使用不同构建目录，避免混淆实际启用的宏。

20.13 读懂现有 benchmark 的统计口径

当前 benchmark 支持按名称选择场景，例如：

- `many_pyramids`、`large_pyramid`。
- `joint_grid`、`compounds`。
- `rain`、`smash`、`spinner`、`tumbler`、`washer`、`junkyard`。

命令参数示例为：

```
--benchmark=many_pyramids --threads=4 --workers=4 --repeats=5 --record-steps
```

这是命令参数示例，不是源码。 `--threads` 是最大线程数上限，实际还会受核数和最大 worker 常量限制。 `--workers` 同样会被限制，应读取输出确认实际运行数。

程序在计时循环之前先执行一个初始 Step，避免初始成本直接进入后面的总 run 时间。 Debug 模式只运行 10 步；Release 使用各场景配置的步数。 因此 Debug 的 10 步不能冒充正式场景成绩。

CSV 不是平均值

当前 `${name}.csv` 写的是同一 worker 多次 run 总耗时的最小值。 如果只测一个 worker，其他未运行 worker 的行可能为零。 这些零不是“这个线程数计算不花时间”。

`--record-steps` 也不是原始逐次样本

`${name}_tN.dat` 的七列依次为：

列	内容
1	`step`
2	`pairs`
3	`collide`
4	`constraints`
5	`transforms`
6	`refit`
7	`sleepIslands`

这些字段通过 `MinProfile` 分别取最小值。 不同列可能来自不同 run，不能把一行解释成同一真实帧的完整分解。 也不能用这些最小值样本计算 p95。

当前 profile 数组只在程序开始初始化一次，多个 worker 或场景循环之间没有重新清空。 为避免混合这项统计，严谨比较时每个进程只选择一个场景和一个 worker 数。 若需要中位数或尾延迟，必须保存每次独立运行的原始耗时或另建明确采样的测试入口。

benchmark 在当前目录写入结果，并覆盖同名文件。 应由读者选择独立结果目录，保留配置和原始日志，不要直接覆盖历史基线。

20.14 优化前后的比较矩阵

一次只改变一项条件，能减少解释歧义。

比较	保持不变	主要检查
1 worker 与多 worker	源码、输入、物理参数	精确回归与耗时
SIMD 与 NONE	算法、场景、worker	结果与吞吐
float 与 double	相对场景与输入	各自确定性、远坐标精度
睡眠开关	场景与输入	醒体数量、误差、维护成本
回收 0 与默认值	场景与输入	更新次数、几何误差、回调行为
CCD 开关	其余配置	质量变化，不只时间变化

关闭 CCD 后变快，并不代表同等碰撞质量下的性能优化。 增大睡眠阈值也可能用“更早停止仿真”换时间，应公开其质量变化。

加速比定义为：

$$S = T_{\mathrm{base}} / T_{\mathrm{new}}.$$

若未来实际测到 8 ms 与 6 ms，可报告 $S = 1.333$。 没有测量时，应留下待测表格，而不是写“数组快一个数量级”或“SIMD 必然四倍”。

推荐验收顺序：

1. 编译与内存安全检查。
2. 物理和几何测试。
3. 确定性、回放与快照测试。
4. 再运行统一条件的性能基准。
5. 最后分析收益是否值得新增复杂度。

20.15 自测与参考答案

1. 100 万米附近 float 的 ULP 是多少？
2. 远坐标差应先转 float 再相减，还是先相减再降精度？
3. 两 worker 分别标记 $\{70, 2\}$、$\{9, 2\}$，提交顺序是什么？

4. 浅状态 hash 是否覆盖全部世界?
5. `--record-steps` 当前输出能直接计算真实 p95 吗?
6. `DeterminismTest` 是否只做 epsilon 比较?
7. 为什么两份克隆互相一致还不够?

参考答案:

1. $2^{19-23} = 0.0625\ \mathrm{m}$。
2. 先以世界坐标精度相减，再转换局部差值。
3. 2、9、70，每个 ID 只处理一次。
4. 不能；deep hash 也有覆盖范围限制。
5. 不能，它按字段保存最小值，不是原始时间样本。
6. 不是，它对当前 golden sleepStep 与 hash 使用精确相等。
7. 两份克隆可能遗漏相同状态，应再与真实原世界未来轨迹比较。

20.16 源码导航

主题	相对源码链接与入口行号
FMA 与构建配置	`CMakeLists.txt`，50、65、116
混合精度与相对差	`math_functions.h`，50、627、649、683
自定义数学函数	`math_functions.c`，96、142
保守世界 AABB	`geometry.c`，405、430、485
接触确定提交	`physics_world.c`，740–883
录制帧末 hash	`physics_world.c`，1034
移动位图稳定消费	`solver.c`，1813
配对串行创建	`broad_phase.c`，469
浅状态 hash	`recording.c`，896
快照与 deep hash	`world_snapshot.c`，43、848、939、966、1021
精确确定性测试	`test_determinism.c`，18、163
大世界与快照测试	`test_large_world.c`，99、308；`test_snapshot.c`，227
测试过滤入口	`main.c`，50；`test_macros.h`，72
benchmark 统计口径	`benchmark/main.c`，40、143、327、375、405

返回 README · 上一章：图着色、SIMD 与多线程